

4028R0

2026-02 Library Evolution Poll Outcomes

Published Proposal, 2026-04-20

Authors:

[Inbal Levi - Library Evolution Chair](#) (Microsoft LTD)

[Fabio Fracassi - Library Evolution Assistant Chair](#) (CODE University of Applied Sciences)

[Andreas Weis - Library Evolution Assistant Chair](#) (ekxide IO GmbH)

[Corentin Jabot - Library Mailing List Review Manager](#)

Source:

[GitHub](#)

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Audience:

WG21

Table of Contents

1 Introduction

2 Poll Outcomes

3 Selected Poll Comments

3.1 Poll 1: Address

3.2 Poll 2: Address US 67-118, PL-012, GB 03-119, DE-120, and FI-121 by applying the changes in "P3842R0: A conservative fix for constexpr uncaught_exceptions() and current_exception()" and forward to LWG for C++26.

3.3 Poll 3: We want to resolve the issues regarding trivial copyability of expected raised by US 135-216, US 136-217 via LWG 4513 issue that applies the suggested changes to C++26.

3.4 Poll 4: Apply the changes discussed (make address_return_t exposition-only, apply wording fix) to P3936R0 and apply as the resolution to FR-030-310 and forward to LWG for C++26.

3.5 Poll 5: Resolve the NB comment US 254-385 by applying the wording from P3980R0 section 3.1 (wording change A amended for member overload) and send to LWG for C++26.

- 3.6 Poll 6: We accept the changes in P3980R0 as the resolution of:
- (1) "US 253-386 33.13.6.5 [task.promise] Allow use of arbitrary allocators for coroutine frame LWG4333" (fixed by applying wording from P3980R0 section 3.1)
 - (2) "US 255-384: 33.13.6.5 [task.promise] Use allocator from receiver's environment LWG4335" (Paper: P3980R0) (by wording changes from P3980R0 section 4.1)
 - (3) US 261-391: 33.13.6.5p3, 16, 17 [task.promise] Bad specification of parameter type (wording removed as part of P3980R0 section 3.1), and forward to LWG for C++26.
- 3.7 Poll 7: Adopt the proposed changes that apply to `inplace_vector` `try_push_back` and `try_emplace_back` from P3981R0 to address US 68-122, US 150-228 and GB 08-225, partially address PL-006, and reject US 149.
- 3.8 Poll 8: Adopt the proposed changes that apply to `inplace_vector` `try_append_range` from P3981R0 to partially address PL-006.

References

Non-Normative References

§ 1. Introduction

In 2026-02, the C++ Library Evolution group conducted a series of electronic decision polls [\[P4027R0\]](#). This paper provides the results of those polls and summarizes the results.

In total, 21 people participated in the polls. Thank you to everyone who participated, and to the papers' authors for all their hard work!

§ 2. Poll Outcomes

- SF: Strongly Favor.
- WF: Weakly Favor.
- N: Neutral.
- WA: Weakly Against.
- SA: Strongly Against.

Poll	SF	WF	N	WA	SA	Outcome
Poll 1: Address NB US 90-197 by adopting the wording changes affecting <code>define_static_object</code> ([<code>meta.syn</code>] and [<code>meta.define.static</code>]) from P3739R4 and forward to LWG for C++26.	2	2	2	3	12	Consensus against

Poll 2: Address US 67-118, PL-012, GB 03-119, DE-120, and FI-121 by applying the changes in "P3842R0: A conservative fix for constexpr uncaught_exceptions() and current_exception()" and forward to LWG for C++26.	8	8	3	0	1	Consensus in favor
Poll 3: We want to resolve the issues regarding trivial copyability of expected raised by US 135-216, US 136-217 via LWG 4513 (later changed into LWG 4026) issue that applies the suggested changes to C++26.	12	4	1	0	0	Strong consensus in favor
Poll 4: Apply the changes discussed (make address_return_t exposition-only, apply wording fix) to P3936R0 and apply as the resolution to FR-030-310 and forward to LWG for C++26.	9	7	1	0	1	Consensus in favor
Poll 5: Resolve the NB comment US 254-385 by applying the wording from P3980R0 section 3.1 (wording change A amended for member overload) and send to LWG for C++26.	5	6	2	0	0	Consensus in favor
Poll 6: We accept the changes in P3980R0 as the resolution of: (1) "US 253-386 33.13.6.5 [task.promise] Allow use of arbitrary allocators for coroutine frame LWG4333" (fixed by applying wording from P3980R0 section 3.1) (2) "US 255-384: 33.13.6.5 [task.promise] Use allocator from receiver's environment LWG4335" (Paper: P3980R0) (by wording changes from P3980R0 section 4.1) (3) US 261-391: 33.13.6.5p3, 16, 17 [task.promise] Bad specification of parameter type (wording removed as part of P3980R0 section 3.1), and forward to LWG for C++26.	3	7	2	0	0	Consensus in favor
Poll 7: Adopt the proposed changes that apply to inplace_vector try_push_back and try_emplace_back from P3981R0 to address US 68-122, US 150-228 and GB 08-225, partially address PL-006, and reject US 149.	11	3	2	1	3	Consensus in favor
Poll 8: Adopt the proposed changes that apply to inplace_vector try_append_range from P3981R0 to partially address PL-006.	1	9	1	1	5	No consensus

All the polls besides Poll 1 and Poll 8 have consensus in favor, and the papers will be forwarded to LWG.

§ 3. Selected Poll Comments

For some of the comments, small parts were removed to anonymize.

§ 3.1. Poll 1: Address

NB US 90-197 by adopting the wording changes affecting `define_static_object` (`[meta.syn]` and `[meta.define.static]`) from P3739R4 and forward to LWG for C++26.

NOTE: The change in votes comparing to the telecon is most likely due to additional discussion and information provided in the reflector (Sent to lib-ext: "[P3739R4 changes to meta::define_static_object \(US 90-197\)](#)")

(...) the original proposal is about static and static and locals are used more than malloc and new. Consistency with numerous guidelines; Looking at a scale the default being reference seems clear.

— Strongly Favor

Agree with the paper

— Weakly Favor

The pointer vs address return part is controversial. I am undecided.

— Neutral

Consistency is more important than "safety" here, and there isn't actually any well-motivated reason to change to T& as return type.

— Weakly Against

The motivation stated in the paper is not applicable, this does not provide any safety benefits as the function in question is consteval. The change introduces undesirable inconsistencies in the interface, as pointed out in the telcon. This would be a clear change for the worse if accepted.

— Strongly Against

`define_static_object` is an allocation function so it should return a pointer.

— Strongly Against

The suggested safety motivation doesn't apply to consteval-only constructs, and making these changes has a quite likely detrimental effect on very plausible use cases of this functionality, in particular construction of statically defined object graphs. In such, it's just better to return a pointer rather than an optional reference.

— Strongly Against

§ 3.2. Poll 2: Address US 67-118, PL-012, GB 03-119, DE-120, and FI-121 by applying the changes in "P3842R0: A conservative fix for constexpr `uncaught_exceptions()` and `current_exception()`" and forward to LWG for C++26.

This fixes an unexpected defect and provides sufficient time to explore the issue in more detail in the 29 time frame.

— Strongly Favor

The original paper clearly needs more implementation experience,

— Strongly Favor

I continue to think that `current_exception` could have been allowed in constexpr as long as it returns an exception from the current evaluation. But this is acceptable.

— Weakly Favor

Isn't the goal for the majority of the language to be constexpr. Also in this release didn't we make many exceptions constexpr. Better to be more strict and reduce the restriction versus making it more strict latter if it constitutes a breaking change.

— Neutral

— Strongly Against

§ 3.3. Poll 3: We want to resolve the issues regarding trivial copyability of expected raised by US 135-216, US 136-217 via LWG 4513 issue that applies the suggested changes to C++26.

We generally favor making as many types trivially copyable as possible. This makes it possible to run more kinds of parallel algorithms on a variety of accelerators, including GPUs, because often the only way to get C++ objects to a place where an accelerator can access them is by copying bytes.

— Strongly Favor

This makes it consistent with related types (optional and variant), and avoids users having to write their own non-standard expected type to get this property.

— Strongly Favor

A reasonable extension of the original design that makes it more consistent with other vocabulary types.

— Weakly Favor

§ 3.4. Poll 4: Apply the changes discussed (make `address_return_t` exposition-only, apply wording fix) to P3936R0 and apply as the resolution to FR-030-310 and forward to LWG for C++26.

I've been arguing against this function returning `T*` since day one, so I'm very happy with this. The `address_return_t` name is inconsistent with library conventions, so I'm glad it will only be exposition only.

— Strongly Favor

If we had `constexpr void*` cast before, this would've been the original design.

— Strongly Favor

It seems that `void*` gives us the best of all options:

- it's not a `T*` (so it's hard for users to misuse it);
- it always exists (unlike `uintptr_t`); and
- it can be used in a constant expression. Idiomatic C++ uses `void*` as a handle type that can be cast to an unsigned integer. That's the type we have and it's the right type, at least for C++26.

— Strongly Favor

Improves API safety

— Weakly Favor

The change seem like good compromise, between `constexpr` support and risk of accidental mutations.

— Weakly Favor

The status quo (return `T`) is *best*. *T* is the correct type to communicate the address of a `T` object, regardless of anything else, period. It's also much simpler to explain and simpler to implement.

— Strongly Against

§ 3.5. Poll 5: Resolve the NB comment US 254-385 by applying the wording from P3980R0 section 3.1 (wording change A amended for member overload) and send to LWG for C++26.

This is much more consistent with how we do allocators elsewhere.

— Strongly Favor

Seems to resolve the issue, and I agree that requiring a specific order is better than supporting arbitrary positions for the allocator args.

— Weakly Favor

Not familiar enough to vote with confidence.

— Neutral

§ 3.6. Poll 6: We accept the changes in P3980R0 as the resolution of:

- (1) "US 253-386 33.13.6.5 [task.promise] Allow use of arbitrary allocators for coroutine frame LWG4333" (fixed by applying wording from P3980R0 section 3.1)
- (2) "US 255-384: 33.13.6.5 [task.promise] Use allocator from receiver's environment LWG4335" (Paper: P3980R0) (by wording changes from P3980R0 section 4.1)
- (3) US 261-391: 33.13.6.5p3, 16, 17 [task.promise] Bad specification of parameter type (wording removed as part of P3980R0 section 3.1), and forward to LWG for C++26.

Useful, last moment we can make this change

— Strongly Favor

I am still not convinced that the overall design of task is sufficiently mature for inclusion in 26, but this change is a clear improvement over the status quo.

— Weakly Favor

Not familiar enough to vote with confidence.

— Neutral

§ 3.7. Poll 7: Adopt the proposed changes that apply to `inplace_vector::try_push_back` and `try_emplace_back` from P3981R0 to address US 68-122, US 150-228 and GB 08-225, partially address PL-006, and reject US 149.

Better than pointer but less consistent with `&` than `optional<&>` would be.

— Strongly Favor

`T*` is just the wrong return type.

— Strongly Favor

That's probably the return type we would have used across the standard library, if we had `optional<T&>` from the get go. It's a shame we can't apply a change like this across the standard library.

— Weakly Favor

I am not convinced that the additional benefit in having `try_push_back` return the position (instead of a plain `bool`) justifies the increase in complexity in the interface.

— Neutral

Unnecessarily complicates the API and results in terrible debugging experience.

— Weakly Against

`optional<T&>` is not an appropriate replacement for `T*` in this context. The return value is not an object (or reference to object, or optional reference to object); it's semantically equivalent to an iterator (pointer).

— Strongly Against

We should adopt the proposed changes from P3981R1 instead.

— Strongly Against

§ 3.8. Poll 8: Adopt the proposed changes that apply to `inplace_vector` `try_append_range` from P3981R0 to partially address PL-006.

NOTE: Comments suggest that the lack of consensus for fixing the interface was due to desire to remove it all together, which was scheduled for discussion later, and agreed upon during Croydon (see [PL-006 Github](#)).

This is an improvement over the status quo.

— Strongly Favor

P4022 is the paper to remove it, but until we process that one, I think returning subrange is better than iterator.

— Weakly Favor

Not feeling strongly about the fix, would probably favour removal

— Weakly Favor

On the one hand, the current `try_append_range` return value is consistent with the C++26 parallel range algorithms. On the other hand, users of the parallel range algorithms would normally address "insufficient output" by reallocating output space (outside of the parallel algorithm) and trying the parallel algorithm again. It's not straightforward to do that with `inplace_vector`, because the capacity is part of the type. Thus, I don't have strong feelings either way. For this reason, I think it's perfectly fine to remove the function for now.

— Weakly Favor

I am more convinced with concerns regarding boolean check.

— Weakly Against

We should instead adopt P4022 and remove `try_append_range`.

— Strongly Against

The issue identified in the meeting where `if(try_append_range(...))` does the wrong thing is a deal-breaker. The function should be removed for C++26. Even if that does not happen, the status quo is preferable over this change for that reason.

— Strongly Against

Unnecessarily complicates the API and results in terrible debugging experience.

— Strongly Against

§ References

§ Non-Normative References

[P4027R0]

Inbal Levi; et al. *2026-02 Library Evolution Polls*. 23 February 2026. URL:
<https://wg21.link/P4027R0>